Chapter 4 — APIs, HTTP & Reliability

4.1 REST, HTTP Semantics and Serving a Model API

Conceptual Deep Dive

REST maps resources to URLs and operations to verbs: `GET` (safe, idempotent), `PUT`/`DELETE` (idempotent), `POST` (neither by default), `PATCH`. Status codes carry meaning: 2xx success, 4xx client fault (400 validation, 401/403 auth, 404, 409 conflict, 422 semantic, 429 rate limit), 5xx server fault (500, 502, 503, 504). LLM endpoints are usually `POST /v1/chat/completions`-style with JSON bodies. Long inference favours **async job** patterns: `POST /jobs` → `202 Accepted` + job id → `GET /jobs/{id}` (or webhook).

Cross-Connection Scenarios

FastAPI + pydantic (3) exposes your RAG/agent service (9, 11); containers/K8s (13) host it; tracing (14) follows request ids.

Production Anti-Patterns

- Returning 200 with an error body.
- No request size limits or timeouts.
- Leaking stack traces and provider keys in errors.

Minimal Working Code

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI()

class ChatIn(BaseModel):
    message: str = Field(min_length=1, max_length=8000)

@app.post("/chat")
async def chat(body: ChatIn):
    if "forbidden" in body.message:
        raise HTTPException(status_code=422, detail="unsupported content")
    return {"reply": f"echo: {body.message}"}
```

4.2 Streaming: Server-Sent Events and WebSockets

Conceptual Deep Dive

SSE is a one-way, text/event-stream over plain HTTP; each event is `data: ...\n\n`; browsers auto-reconnect (EventSource). It fits token streaming and passes through most proxies if buffering is disabled (`X-Accel-Buffering: no`). **WebSocket** is full-duplex over an upgraded TCP connection: use for voice/realtime, collaborative or interrupt-able agents. Trade-offs: SSE is simpler, cacheable-friendly and works with HTTP/2; WebSockets need sticky sessions, heartbeat (ping/pong) and custom reconnection.

Cross-Connection Scenarios

Perceived-latency gains (2.4); backpressure matters for serving (13); disconnect handling ties to cancellation in asyncio (3).

Production Anti-Patterns

- Proxy buffering that makes streams arrive in one lump.
- Not stopping generation when the client disconnects (paying for unread tokens).
- No heartbeat → idle connections dropped by load balancers.

Minimal Working Code

```
import asyncio
from fastapi import FastAPI, Request, WebSocket
from fastapi.responses import StreamingResponse

app = FastAPI()

async def tokens(prompt: str):
    for t in f"streaming reply to {prompt}".split():
        yield t + " "
        await asyncio.sleep(0.05)

@app.get("/sse")
async def sse(prompt: str, request: Request):
    async def gen():
        async for t in tokens(prompt):
            if await request.is_disconnected():
                return
            yield f"data: {t}\n\n"
        yield "data: [DONE]\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream",
                             headers={"X-Accel-Buffering": "no", "Cache-Control&
quot;: "no-cache"})

@app.websocket("/ws")
async def ws(sock: WebSocket):
    await sock.accept()
    while True:
        msg = await sock.receive_text()
        async for t in tokens(msg):
            await sock.send_text(t)
```

4.3 Retries, Exponential Backoff and Jitter

Conceptual Deep Dive

Transient faults (429, 502, 503, 504, timeouts, connection resets) deserve retries; 400/401/403/422 do not. **Exponential backoff** waits base * 2^attempt, capped; **jitter** randomises the wait to avoid synchronised retry storms ("full jitter": sleep = random(0, min(cap, base*2^n))). Honour the Retry-After header. Set a retry **budget** and a total deadline so retries cannot amplify an outage (retry amplification multiplies across layers).

Cross-Connection Scenarios

Pairs with circuit breakers (4.4) and idempotency (4.5); evaluation harnesses (10) need it to survive rate limits; provider fallback ties to 2.2.

Production Anti-Patterns

- Retrying non-idempotent calls without keys.
- Retries at every layer (3×3×3 = 27 calls).
- Fixed sleep with no jitter.

Minimal Working Code

```
import asyncio
import random
import httpx

RETRYABLE = {429, 500, 502, 503, 504}

async def post_with_retry(client: httpx.AsyncClient, url: str, json: dict,
                          max_attempts: int = 5, base: float = 0.5, cap: float = 20.0):
    for attempt in range(max_attempts):
        try:
            r = await client.post(url, json=json, timeout=30)
            if r.status_code not in RETRYABLE:
                r.raise_for_status()
                return r.json()
            retry_after = r.headers.get("Retry-After")
            delay = float(retry_after) if retry_after and retry_after.isdigit() \
                else random.uniform(0, min(cap, base * 2 ** attempt))
        except (httpx.TimeoutException, httpx.TransportError):
            delay = random.uniform(0, min(cap, base * 2 ** attempt))
        if attempt == max_attempts - 1:
            raise RuntimeError("retries exhausted")
        await asyncio.sleep(delay)
```

4.4 Rate Limiting and Circuit Breakers

Conceptual Deep Dive

Providers limit **RPM** (requests/min) and **TPM** (tokens/min). Client-side limiters keep you under: **token bucket** (refill rate r, capacity b; allows bursts) and **sliding window**. Server-side, limit per API key/user to protect cost. A **circuit breaker** has three states: *closed* (normal), *open* (fail fast after N failures within a window), *half-open* (probe after cooldown; success closes, failure re-opens). It stops you hammering a dead dependency and enables fast fallback.

Cross-Connection Scenarios

Fallback to a second provider or cached answer (2.2, 9); metrics for breaker state go to observability (14).

Production Anti-Patterns

- Limiting only requests, not tokens.
- Per-process limiters in a multi-replica deployment (use Redis).
- Breaker with no fallback path.

Minimal Working Code

```
import time

class TokenBucket:
    def __init__(self, rate: float, capacity: float):
        self.rate, self.capacity = rate, capacity
        self.tokens, self.ts = capacity, time.monotonic()

    def allow(self, cost: float = 1.0) -> bool:
        now = time.monotonic()
        self.tokens = min(self.capacity, self.tokens + (now - self.ts) * self.rate)
        self.ts = now
        if self.tokens >= cost:
            self.tokens -= cost
            return True
        return False

class CircuitBreaker:
```

```
    def __init__(self, threshold=5, cooldown=30.0):
        self.threshold, self.cooldown = threshold, cooldown
        self.failures, self.opened_at = 0, None

    def call(self, fn, *a, **kw):
        if self.opened_at is not None:
            if time.monotonic() - self.opened_at < self.cooldown:
                raise RuntimeError("circuit open")
            self.opened_at = None  # half-open: allow one probe
        try:
            out = fn(*a, **kw)
        except Exception:
            self.failures += 1
            if self.failures >= self.threshold:
                self.opened_at = time.monotonic()
            raise
        self.failures = 0
        return out
```

4.5 Idempotency and Asynchronous Request Handling

Conceptual Deep Dive

An operation is **idempotent** if repeating it has the same effect as once. Retries are only safe for such operations. For POST, clients send an Idempotency-Key; the server stores key → result for a TTL and replays it on duplicates. Async patterns: background tasks, queues (Celery, RQ, SQS, Kafka), the *outbox* pattern, webhooks with signed payloads and retry, and polling with 202. Agents calling side-effecting tools (send email, charge card) need idempotency keys per tool call.

Cross-Connection Scenarios

Agent tool execution (11), batch embedding jobs (8), eval runs (10).

Production Anti-Patterns

- Double-charging because a timeout triggered a retry.
- Idempotency keys that are not scoped to user/endpoint.
- Doing long work inside the request thread.

Minimal Working Code

```
import hashlib
import json
import time

class IdempotencyStore:
    def __init__(self, ttl=3600):
        self.ttl, self.data = ttl, {}

    def run(self, user: str, key: str, payload: dict, fn):
        k = (user, key, hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest())
        hit = self.data.get(k)
        if hit and time.time() - hit[0] < self.ttl:
            return hit[1]
        result = fn(payload)
        self.data[k] = (time.time(), result)
        return result

store = IdempotencyStore()
print(store.run("u1", "abc", {"amt": 5}, lambda p: {"charged": p[
"amt"]}))
print(store.run("u1", "abc", {"amt": 5}, lambda p: {"charged": 99
```

```
9}))  # replayed
```

4.6 Timeouts, Deadlines, Caching and Webhooks

Conceptual Deep Dive

Set connect, read and total timeouts separately; propagate a **deadline** downward so inner calls know the remaining budget. Cache deterministic requests (hash of model + params + prompt) in Redis; **semantic caching** matches near-duplicate queries by embedding similarity (risky if threshold is loose). Webhooks need HMAC signature verification and replay protection (timestamp tolerance).

Cross-Connection Scenarios

Semantic cache uses embeddings (8); signature verification relates to security (14).

Production Anti-Patterns

- No timeout (default is often infinite).
- Caching responses that contain user-specific data under a shared key.
- Verifying webhook signatures with `==` (timing attack) instead of `hmac.compare_digest`.

Minimal Working Code

```
import hashlib
import hmac
import time

def verify_webhook(secret: bytes, body: bytes, sig_hex: str, ts: int, tol: int = 300) -> bool:
    if abs(time.time() - ts) > tol:
        return False
    expected = hmac.new(secret, f"{ts}.".encode() + body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sig_hex)

def cache_key(model: str, params: dict, prompt: str) -> str:
    import json
    blob = json.dumps({"m": model, "p": params, "q": prompt}, sort_keys=Tru
e)
    return hashlib.sha256(blob.encode()).hexdigest()
```

4.7 Authentication, Authorization and Multi-Tenancy for AI APIs

Conceptual Deep Dive

Authenticate callers with API keys (hashed at rest, prefix-identifiable, revocable) or OAuth2/OIDC JWTs (verify signature, `exp`, `aud`, `iss`). Authorize per resource: a user must only reach their own conversations, documents and vector namespaces. Multi-tenancy needs per-tenant rate limits, quotas, budgets and data isolation. Add request ids and propagate them to logs and traces. Edge cases: confused-deputy bugs where a tool call runs with the service's rights rather than the user's; key rotation windows; leaked keys in client-side code; streaming responses that continue after a key is revoked.

Cross-Connection Scenarios

ACL-filtered retrieval (8.5), agent permissions (11.6), spend limits (14.2).

Production Anti-Patterns

- Storing API keys in plaintext.
- One shared service credential for every tenant's tool calls.

- Authorization checked in the UI but not the API.

Minimal Working Code

```
import hashlib
import hmac
import secrets

def new_key() -> tuple[str, str]:
    raw = "sk_" + secrets.token_urlsafe(32)
    return raw, hashlib.sha256(raw.encode()).hexdigest()    # store only the hash

def verify(raw: str, stored_hash: str) -> bool:
    return hmac.compare_digest(hashlib.sha256(raw.encode()).hexdigest(), stored_hash)

raw, h = new_key()
print(verify(raw, h), verify("sk_wrong", h))
```

```
import hashlib
from fastapi import Depends, FastAPI, Header, HTTPException

app = FastAPI()
KEYS = {"hash-of-key-1": {"tenant": "acme", "rpm": 60}}

def tenant(authorization: str = Header(...)):
    h = hashlib.sha256(authorization.removeprefix("Bearer ").encode()).hexdigest()
    if h not in KEYS:
        raise HTTPException(status_code=401, detail="invalid key")
    return KEYS[h]

@app.get("/docs/{doc_id}")
def get_doc(doc_id: str, t=Depends(tenant)):
    if not doc_id.startswith(t["tenant"] + ":"):
        raise HTTPException(status_code=404)       # 404, not 403, to avoid leaking existence
    return {"id": doc_id}
```

4.8 Batch APIs, Queues and Backfills

Conceptual Deep Dive

Large offline jobs (embedding a corpus, running evals, labelling data) should use provider **batch endpoints** (often discounted, with a longer completion window) or your own queue (SQS, Redis Streams, Kafka, Celery). Design jobs to be **idempotent**, **resumable** (checkpoint progress), and **observable** (processed/failed counters). Use a dead-letter queue for poison messages after N attempts. Edge cases: duplicate delivery (at-least-once semantics), out-of-order completion, partial batch failures, and result files exceeding memory.

Cross-Connection Scenarios

Embedding ingestion (8.6), eval runs (10), fine-tuning data generation (12.2).

Production Anti-Patterns

- Re-running an entire backfill after one failure.
- No dead-letter queue, so one bad record blocks the pipeline.
- Using synchronous real-time endpoints (full price, tight rate limits) for overnight work.

Minimal Working Code

```
import json
import os
```

```
class Checkpoint:
    def __init__(self, path="progress.json"):
        self.path = path
        self.done = set(json.load(open(path))) if os.path.exists(path) else set()

    def mark(self, item_id: str):
        self.done.add(item_id)
        json.dump(sorted(self.done), open(self.path, "w"))

def process_all(items, fn, ck: Checkpoint, max_attempts=3, dlq=None):
    dlq = dlq if dlq is not None else []
    for it in items:
        if it["id"] in ck.done:
            continue
        for attempt in range(max_attempts):
            try:
                fn(it)
                ck.mark(it["id"])
                break
            except Exception as e:
                if attempt == max_attempts - 1:
                    dlq.append({"item": it, "error": str(e)})
    return dlq
```

```
def chunked(seq, n):
    for i in range(0, len(seq), n):
        yield seq[i:i + n]

def build_batch_requests(prompts, model="MODEL_ID"):
    """JSONL-style records in the shape many provider batch APIs expect (custom_id + body)
."""
    return [{"custom_id": f"req-{i}", "method": "POST", "url
": "/v1/chat/completions",
             "body": {"model": model, "messages": [{"role": "
;user", "content": p}]}}
            for i, p in enumerate(prompts)]

print(len(list(chunked(build_batch_requests(["a", "b", "c"]), 2))))
```